Coursework Report

This project implements a simple baseline for converting videos to panoramas. The main function is to read video files from a folder containing multiple videos, then process each video file into a panorama and automatically correct the rotation state of the panorama.

Main Objectives

Automated video panorama synthesis: Develop an efficient automated script to batch read video files from a folder containing .MOV format videos and automatically process them into panoramic images, reducing manual intervention and improving the efficiency of panorama production.
Achieve high-quality panorama: By applying multiple technologies such as key frame extraction, SIFT feature extraction and matching, and RANSAC algorithm to calculate homography matrix, the relationship between video frame images is accurately processed to generate high-quality, seamlessly stitched panoramic images, improving the visual effect and accuracy of panoramic images.

Key Functionalities Implemented

Video file batch reading: The program can automatically scan the specified folder, identify and read all .MOV format video files in it, and realize batch processing of video data without manually importing videos one by one.

Key frame extraction: Use efficient algorithms to intelligently extract representative key frames from the video, effectively reduce the amount of data for subsequent processing, and ensure that the key frames contain the core information of the video scene, providing necessary materials for panorama synthesis.

SIFT feature extraction and matching: Use the SIFT algorithm to detect and describe the feature points of the extracted key frames, accurately extract the local features of the image, and then perform feature matching between different key frames to find the corresponding relationship between images, providing a reliable basis for subsequent image stitching.

RANSAC algorithm calculates the homography matrix: The built-in RANSAC algorithm is used to optimize the feature matching results, effectively eliminate mismatched points, robustly calculate the homography matrix between images, accurately describe the perspective transformation relationship between images, and improve the accuracy and stability of panorama stitching.

Automatic generation of panoramic images: Based on the calculated homography matrix, the key frames are transformed and fused, and image stitching is automatically completed to generate a complete panoramic image, realizing one-stop processing from video to panoramic images.

Features

Rotation correction of synthesized panoramas

Based on the panorama synthesis function, this project further introduces the classic target detection network technology to realize the rotation correction function of the synthesized panorama.

The starting point for introducing this function is that the result of direct panorama stitching may be rotated and inverted: this may be because there is a problem when matching the first and second photos, resulting in such homography transformation of subsequent pictures. Therefore, it is necessary to determine whether the picture is inverted and rotate the inverted picture back to the right.

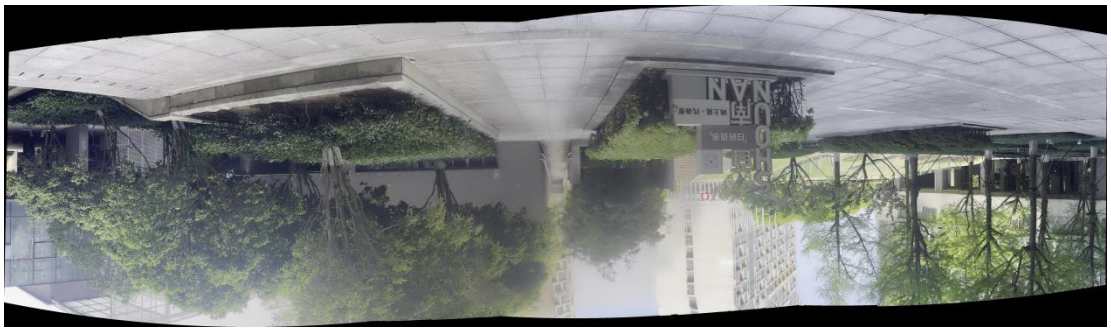

Figure 1: Inverted panoramas produced during stitching.

This function uses a lightweight pre-trained target detection model (YoloV5-S) to identify key objects in the panorama and count the number. Because the target detection network (or convolutional neural network) is not rotationally invariant like the feature operator, the images before and after rotation can be input into the network to obtain the number of targets to determine whether there is a rotation deviation in the panorama. Once it is detected that the number of targets increases after rotating 180 degrees, the panorama is rotated to ensure the correctness of the image direction.

Figure 2: Rotation-corrected panorama.

Panorama black edge cropping

As can be seen from Figure 1, there are many black edges around the synthesized panorama, which is because the hand cannot be completely stable during handheld shooting.
Based on this, I simply cropped the stitched panorama to reduce the area of the black edges and improve the look of the picture.
The main function is to detect and remove the black edges in the image, leaving only the colored center area. Use the OpenCV library to read the image, then detect the maximum width of the black edges around each image, and finally calculate the rectangular area without black edges in the middle based on these widths and crop and save it.

Figure 3: Panorama before black edge removal.

Figure 4: Panorama after black edge removal.

Detailed Steps

Keyframes Extraction

In the key frame extraction part, an intuitive and easy-to-implement extraction strategy is adopted: by setting a fixed interval, the video is uniformly sampled, that is, after each set number of frames, one frame is selected as the key frame. The process of this method is concise and clear, without the need for complex algorithm logic and computing resources. It can complete the key frame extraction by relying on basic counting and frame selection operations, and quickly obtain a certain number of representative frames from the video to meet the initial material requirements of panorama synthesis.

Features Matching

To stitch the images together in order, it is necessary to extract and match the feature points in the images.

First, the two input color images are converted into grayscale images. This is because feature detection is usually performed on grayscale images, which can reduce the amount of calculation and improve the stability of the features.

The SIFT (Scale-Invariant Feature Transform) algorithm is used to detect key points and descriptors in the image. The SIFT algorithm first constructs a Gaussian difference pyramid of the image, and detects key points in the image by finding extreme points in different scale spaces. These key points can still exist stably when the image undergoes scaling, rotation, and other changes. Subsequently, the gradient direction histogram of the pixels around the key point is calculated with the key point as the center, thereby generating a 128-dimensional feature descriptor. These descriptors not only accurately depict the local features around the key point, but also have high robustness to image rotation, scaling, brightness changes, and perspective changes, so that even if there are large differences between different images, they can be quickly and accurately matched through SIFT feature descriptors, playing an important role in many fields such as image stitching, target recognition, and image retrieval.

The FLANN (Fast Library for Approximate Nearest Neighbors) matcher is used for feature matching. FLANN is an efficient nearest neighbor search algorithm widely used in the field of computer vision. It is used to optimize the matching process of SIFT feature points in this project. It divides the feature point data in high-dimensional space into hierarchical layers by constructing a KD tree index structure (set to 5 trees in this project), and organizes the data in a tree structure, so that when looking for the nearest neighbors of feature points, the search range can be quickly narrowed to avoid global traversal, thereby greatly improving the matching efficiency. This structure is particularly suitable for processing a large amount of feature point data. In the feature matching task of complex images, it can not only ensure the accuracy of matching, but also significantly reduce the calculation time, providing fast and reliable feature matching results for subsequent homography matrix calculation and panorama stitching, and enhancing the performance of the entire panorama synthesis process.

Use ratio test to filter out high-quality matching point pairs. When the distance ratio between the best match and the second best match is less than 0.7, it is considered a reliable match. This method can effectively filter out unstable matches.

Finally, the coordinate information of the matching points is extracted and converted into coordinates in list form.

Images Stitching

First, the input is checked. If there are less than 2 input images, the first image is directly returned as the result. This is a fallback strategy.

Use OpenCV's Stitcher class to create a stitcher object and specify the mode as PANORAMA (panoramic mode), which is specifically used to process continuous scenes in the horizontal or vertical direction.

Call the stitcher.stitch() method to perform the actual stitching operation. This method automatically completes a series of complex steps such as feature extraction, feature matching, image transformation, and image fusion.

OpenCV's Stitcher uses the RANSAC algorithm (Random Sample Consensus) at the bottom layer, which is an important tool in the field of computer vision and image processing. When dealing with problems such as image feature matching with a lot of noise and abnormal data, it randomly selects a small amount of sample data, assumes that these samples conform to the ideal model and calculates the model parameters, and then uses the model to test the remaining data and count the number of data points that conform to the model (i.e., the number of inliers). After multiple iterations, the model parameters with the largest number of inliers are retained as the final result, so as to effectively eliminate abnormal data such as mismatched points, accurately calculate models such as homography matrices that describe the transformation relationship between images, and ensure the reliability of data and the accuracy of models in tasks such as panorama stitching and 3D reconstruction.

Other miscellaneous post-processing

After obtaining the stitched panorama, it is input into the target detection network to determine the rotation direction, and then decide whether to rotate 180 degrees based on the detection result.

Finally, both images are saved in the specified directory.

In addition, there is a separate .py file for image cropping.
First, the color image is converted to grayscale, and then the image edge is scanned from four directions (top, right, bottom, and left). The average value of each row/column pixel is calculated and compared with the threshold (currently set to 135) to determine whether it is a black edge.

Scan from top to bottom to detect the height of the top black edge, from bottom to top to detect the bottom black edge, from left to right to detect the left black edge, and from right to left to detect the right black edge. Finally, the cropping length in each direction is limited (no more than 300 pixels at the top/bottom, no more than 100 pixels at the left and right) to ensure that the image content area is not over-cropped.

Experiments

I captured several videos to test the functional integrity of the system. All videos are shot from fixed point and handheld.
The previous article has shown two panoramic views of the video, and three more sets will be shown next.

Figure 5: Panorama of 3.mov

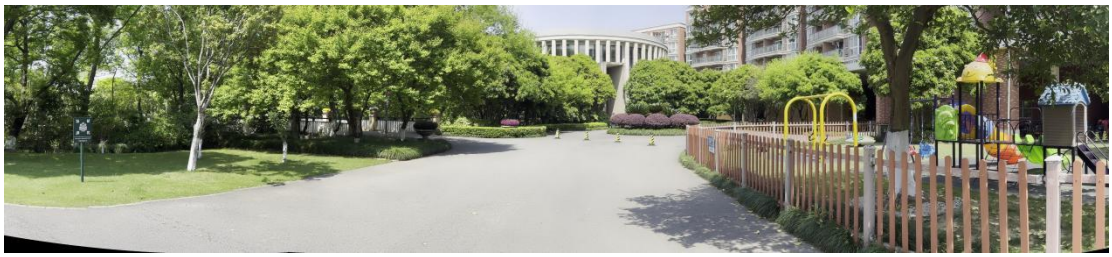

Figure 6: Panorama of 1.mov

Figure 7: Panorama of 5.mov

It is quite obvious that although black edge interception and rotation correction are applied, there are still more black corners in the left and right pictures. This is because the focal length of the camera is short when shooting, resulting in more distortion of the edge images, while the images near the middle section are synthesized from multiple perspectives, so the proportion of black edges is much smaller, that is, the perspective is larger.

Another part of the black edges may be caused by the instability of handheld shooting, which causes most of the pictures to not be on the same horizontal line.

Discussion

In the K**eyframe Extraction** stage, I used a fixed interval method, which is very fast. However, this fixed interval sampling method also has obvious defects. When the video content changes

dynamically, the scene switches frequently, or the object moves at an uneven speed, the key frames extracted at a fixed interval may not accurately capture key actions, important scene transitions, or detailed information. Either key information will be missed, resulting in missing content and incomplete information in the panorama, or a large number of redundant frames will be introduced, increasing the computational burden of subsequent processing, while reducing the quality and efficiency of panorama synthesis.

In **Features Matching** stage, SIFT can generate feature descriptors that are highly robust to rotation, scaling, and illumination changes, ensuring that key points in the image can be stably detected under different conditions. FLANN significantly speeds up feature matching through index structures such as KD trees, making it possible to efficiently complete matching in large-scale feature data sets. The combination of the two can achieve accurate and fast feature point matching in complex scenes.

However, this combination also has some disadvantages. The SIFT algorithm has a high computational complexity and consumes more computing resources and time, resulting in a relatively slow feature extraction process. At the same time, FLANN uses an approximate nearest neighbor search, which may introduce certain matching errors in some scenarios with extremely high matching accuracy requirements. In addition, the SIFT descriptor has a high dimensionality and takes up more storage space, which increases the cost of data processing to a certain extent.

The underlying layer of the **Images Stitching** stage is RANSAC, which is very robust to outliers and can accurately estimate model parameters in the presence of a lot of noise and mismatches. The principle is simple and easy to implement. However, the algorithm has low computational efficiency, especially when the amount of data is large. Multiple iterations will consume a lot of time and computing resources, and the final result may not be the global optimal solution.

About **Rotation Correction**, after testing, this function can effectively solve the angle offset problem caused by the stitching process of the panorama, improve the standardization and usability of the panorama, and further enhance the project's automated processing capabilities, making the entire process from video to high-quality panorama more complete and intelligent.

Of course, from the perspective of practical application, **such a method may also have a certain possibility of misjudgment**, and it is necessary to adjust the parameters according to the actual situation to adapt to the target scenario.

As for **Black Edge Cropping**, I also implemented a relatively direct method, which can solve the problem when the perspective offset is within a certain range. However, if the perspective moves too much, it may not be possible to generate an ideal rectangular image in the end.

Conclusion

The method I implemented can quickly process handheld videos shot in situ into panoramic images. Its **advantages** are:
- The data requirements are simple and can be generated by handheld shooting with a mobile phone;
- The generation speed is relatively fast, the operation is simple, and no camera calibration is required.

But there are also certain **disadvantages:**
- It is impossible to give a standard rectangular range image with certainty, that is, there will be some black edges;
- It cannot process moving images.

Overall, it has achieved my expected results. But there are also some places that may be realized but have not yet been realized, such as for the black edges on the edges, can affine transformation be used to optimize the filling? Or use a more robust rotation detection algorithm. This project still has a lot of room for development.